



A comprehensive guide for creating modkits in build island / bild iland.

Guide Created By: [@prowrecker145](#)

Before You Start

Enable Deprecated Objects

Bild Iland uses many deprecated objects due to its older codebase. You'll need to enable these in Studio:

1. Open Roblox Studio
2. Click on **FILE** in the top menu
3. Select **Studio Settings**
4. In the left sidebar, click on **Studio**
5. Scroll down and check the box for **Show Deprecated Objects in Explorer**
6. Click **OK** to save

Prerequisites

This guide assumes you have basic scripting knowledge:

- Variables and value types
- Navigating Roblox Studio Explorer and Properties
- Understanding of CFrames and part sizing
- Basic Lua syntax

If you're new to scripting, consider reviewing Roblox's scripting tutorials first at: <https://create.roblox.com/docs/tutorials/scripting/basic-scripting>

Modkit Structuring Basics

There are two main templates available for creating modkits. The first is the original version used in **Build Island**, and the second is a reworked, improved version used in **Bild Iland**. This guide explains how to set up your folders and code for both.

It is highly recommended to use the new Bild Iland Template

Bild Iland Template (V2)

 joekit.rbxm

The **Modkit Module** is the heart of your mod. It acts as a container for all your folders and assets. Make sure the name of this ModuleScript is exactly what you want your modkit to be called. Typically any extra assets that your blocks or other scripts need to be functional will be placed in the Modkit Module directly and reparented when the modkit is loaded.

Folder Structure:

- **Modkit Module** (ModuleScript named after your mod)

- **Assets** (Folder) — Put your blocks/items here.
- **Tools** (Folder) — Put tools here for lighting/admin access.

Module Code:

```
--[[ Build Island ModKit V2 by Toybox Development
roblox.com/groups/7712711/Toybox-Development#!/about
Build Island ModKit V2 made by Blockdown64
]]

-- The modkit name is the name of this ModuleScript

local modkit = {
    description = "My first Build Island mod!"; -- Description of the
modkit
    thumbnail = "rbxassetid://13832154002"; -- Thumbnail that will appear
in the import menu
    credit = { -- User IDs of the creator(s) of the modkit
        421267083;
    }
}

-- This function is called when the mod is loaded
function modkit.Load(mods)
    -- You can add code here to run when the mod starts
end

-- This function is called when the mod is unloaded
function modkit.Unload(mods)
    -- You can add code here to run when the mod is removed
end

return modkit

-- NOTE: TO SUBMIT MODKITS YOU MUST BE AT LEAST 13 YEARS OF AGE!
```

Build Island Template (V1)

<https://www.roblox.com/library/6160073881/ModKit>

In the original version, the main container is a Model named with a hashtag (e.g., #MyMod). The script and folders are placed inside that model. Any extra assets that your blocks or other scripts

need to be functional will be placed in the #mymod module and reparented when the modkit is started.

Folder Structure:

- **ModName (Model)**
 - a. **#mymod (ModuleScript)** — This contains your settings and start function. You replace mymod with your modkit name. Ex: #protilities
 - b. **Items (Folder)** — Put your blocks here.
 - c. **Gears (Folder)** — Put tools here. It will put your tools on lighting.
 - d. **Commands (Folder)** — Put custom command modules here. (This is unused and has never been implemented)

Modkit Module V1:

```
--[[ Build Island ModKit by Toybox
roblox.com/groups/7712711/Toybox-Development#!/about
]]

local mymod = {
    description = "My first Build Island mod!";
    credit = "Toybox Development";
    v = 1;
}

-- This function is called when the mod is loaded
function mymod.start(mods)
    --[[ mods is a table that contains all of the loaded mods
    ex:
    local yourmod = mods["#yourmod"]
    local description = yourmod.description
    ]]
end

return mymod
```

Block Structure Basics

Every block in bild iland follows this structure:

Required Components

1. Main Model

- Name this whatever you want your block to be called (e.g., "Door", "Button", "Vehicle")

2. AssetId (IntValue) ⚠ MANDATORY

- Place this inside the main model
- Must be a random number smaller than 2^{53}
- **Recommended length: 4-9 digits** (e.g., 12345678)
- Each block **MUST** have a unique AssetId
- ⚠ **CRITICAL:** If two blocks share the same AssetId, your entire modkit will break
- ⚠ **WARNING:** AssetIds that are too large can make blocks unsaveable and cause severe issues

3. Model (Model or BasePart)

- **Must be named exactly "Model"** or the block won't be placeable
- Place this inside the main model
- Contains all the physical parts of your block
- **If it's a BasePart:** Players can resize the block in-game
- **If it's a Model:** The block cannot be resized

Basic Structure:

```
YourBlockName (Model)
├── AssetId (IntValue) - Example: 12345678
├── Model (Model or BasePart)
│   └── [All your block's parts go here]
```

Optional Block Components

Add these IntValues inside your main model to customize behavior:

Rank (IntValue)

- Determines minimum rank required to place this block
- Values: **0 to 5**
- Default: 0 (anyone can place it)

Budget (IntValue)

- Limits how many of this block non-admins can place (Rank 3 and below)
- Useful for performance-intensive blocks

SubtypeOfId (IntValue)

- Contains the AssetId of the folder block this belongs to
- See **Block Organization & Subcategories** section

Semantic (BoolValue)

- Must be set to **false**
- Required for blocks used as category folders
- See **Block Organization & Subcategories** section

Example Advanced Structure:

YourBlockName (Model)

└─ AssetId (IntValue) - Example: 12345678

└─ Rank (IntValue) - Example: 4

└─ Budget (IntValue) - Example: 15

└─ SubtypeofId (IntValue) - Example: 87654

└─ Semantic (BoolValue) - Example: true

└─ Model (Model or BasePart)

└─ OwnerId (IntValue) - This contains the User ID of the player who placed the block. This gets created upon placing the block and should not be in your block when creating your modkit.

└─ [All your block's parts go here]

Block Type Model

Multi-part blocks / Models (doors, vehicles, interactive objects) typically include:

- A Primary hitbox part
- Visual parts (meshes, unions, basic parts)

- Welds connecting everything
- Scripts for functionality

The Primary Part - Critical Requirements

The Primary part is the foundation of your block's placement system.

Setup Requirements:

1. **Name:** Must be called "Primary"
2. **Transparency:** Set to 1 (invisible)
3. **CanCollide:** Set to false
4. **Size:** Must be a multiple of 4 on ALL axes (X, Y, Z)
 -  Valid: 4×4×4, 8×4×12, 16×20×4
 -  Invalid: 5×4×4, 8×6×12, 10×10×10
5. **Set as PrimaryPart:** In the Model's properties, set PrimaryPart to your Primary part
6. **Recommended:** Contain all other parts within the Primary part's boundaries

Why this matters: Build and places blocks on a 4×4×4 grid. If your Primary part doesn't follow these rules, the game creates its own hitbox, causing placement issues.

Welding and Physics

For movable blocks (vehicles, ragdolls, physics objects):

- Weld all parts together
- Unanchor all parts
- Recommended tool: **DWeld plugin** (automatically welds everything to PrimaryPart)

For static blocks (buildings, decorations):

- Parts can remain anchored
- Welding not required

Block Type Parts / Single Instances

For simple blocks made of one BasePart named "Model":

- The part automatically unanchors when placed
- This behavior cannot be changed
- **To keep it anchored:** Put the BasePart inside a Model instead
 -  Note: This makes the block non-resizable

Block Organization & Subcategories

In the stamper menu, some blocks have a blue background. Clicking these opens a submenu with more blocks. These are called **subcategories** and help organize your modkit.

Creating a Folder Block

1. Follow standard block structure
2. Add these additional values to the main model:
 - **SubtypeOfId (IntValue)**: Set to the same value as this block's AssetId
 - **Semantic (BoolValue)**: Set to false
3. Make the visual simple (you'll never place this block)
4. This block becomes a non-placeable folder button

Adding Blocks to a Subcategory

1. Create your block normally
2. Add a **SubtypeOfId (IntValue)** to the block's main model
3. Set its value to match the folder block's AssetId

Done! Your block now appears in that subcategory.

Organization Notes

- Blocks without SubtypeOfId appear in the main modkit category
- Blocks appear in the order they're placed in the Assets folder
- **For alphabetical sorting:** Manually arrange them or create a sorting script (recommended for large modkits)

Block Configuration Guide

The config tool lets players customize your blocks. Here's how to make blocks configurable.

Creating a Configuration Object

1. Select the Model/BasePart that contains your Primary part
2. Click the + button in the Explorer
3. Search for "Configuration" and insert it
4. It must be named exactly "**Configuration**"

Now you can add settings!

Basic Setting Types

BoolValue

- True/false toggle
- Simple on/off settings

StringValue

- Text input
- Automatically filtered by bild iland
- Examples: Custom text on signs, player names

IntValue

- Whole numbers only
- Decimals automatically round
- Examples: Speed values, counts

NumberValue

- Supports decimals
- Examples: Precise measurements, multipliers

IntConstrainedValue

- Integer with minimum and maximum limits
- Examples: Volume (0-10), brightness (0-100)

DoubleConstrainedValue

- Decimal number with min/max limits
- Examples: Precise ranges with decimal control

Advanced Setting Types

ObjectValue

- Lets players select and reference objects
- Examples: Teleporter destinations, linked blocks

Dropdown Menus

1. Create a **StringValue** with your setting name
2. Add **BoolValues** inside the StringValue
3. Name each BoolValue after an option
4. **Set all BoolValues to true** (or they won't appear!)
5. Set the StringValue's value to the default option name

Example structure:

WeaponType (StringValue) - Value: "Sword"

```
|— Sword (BoolValue) - true
|— Bow (BoolValue) - true
|— Staff (BoolValue) - true
```

Bild Iland Exclusive Settings

These only work in bild iland:

Vector3Value

- Stores X, Y, Z coordinates
- Examples: Position offsets, spawn points

Color3Value

- RGB color picker with color wheel interface
- Examples: Light colors, part colors

BrickColorValue

- Premade color palette
- Classic Roblox color picker

Wiring Basics

Wiring allows blocks to communicate and enables automation. It uses two main objects:

CustomEvent (Output)

Sends signals to other blocks. Learn more:

<https://create.roblox.com/docs/reference/engine/classes/CustomEvent>

Basic usage:

```
CustomEvent:SetValue(1) -- Turn on (sends signal)
CustomEvent:SetValue(0) -- Turn off
```

CustomEventReceiver (Input)

Receives signals from CustomEvents. Learn more:

<https://create.roblox.com/docs/reference/engine/classes/CustomEventReceiver>

Basic usage:

```
local Input = script.Parent.Input

Input.SourceValueChanged:Connect(function(value)
    if value >= 0.5 then
        -- Block is powered ON
        print("Block activated!")
    else
        -- Block is powered OFF
        print("Block deactivated!")
    end
end)
```

Important Wiring Notes

- CustomEventReceivers can only store ONE CustomEvent at a time
- One output can connect to multiple inputs
- One input can only connect to one output

Traditional Gate Lighting

Gates typically show their powered state with indicator lights:

- **Red:** Powered off
- **Green:** Powered on

Example code:

```
local base = script.Parent
local Input = base.WaitForChild("Input")
```

```

local light1 = base:WaitForChild("Light1") -- Input light
local light2 = base:WaitForChild("Light2") -- Output light

Input.SourceValueChanged:Connect(function(value)
    if value >= 0.5 then
        light1.BrickColor = BrickColor.Green()
    else
        light1.BrickColor = BrickColor.Red()
    end
end)

```

Wiring Identifiers

Identifiers let your blocks target specific players. **You must code these manually** - the game doesn't handle them automatically.

@m - Last Interacted Player

Targets the player who last interacted with the input source.

Setup: Add an ObjectValue called "LastInteractedWith" inside your CustomEvent (output).

Input code example:

```

local input = script.Parent.Input

input.SourceValueChanged:Connect(function(value)
    if value >= 0.5 then
        local isPlayerInteractable =
input.Source:FindFirstChild("LastInteractedWith")
        local interactedPlayer = isPlayerInteractable and
isPlayerInteractable.Value
        if interactedPlayer then
            print("Last player:", interactedPlayer.Name)
        end
    end
end)

```

Output code example:

```

local output = script.Parent.Output

game.Players.PlayerAdded:Connect(function(player)
    output.LastInteractedWith.Value = player
    output:SetValue(1)
    task.wait(1)
    output:SetValue(0)
end)

```

@a - All Players

Loops through every player in the game when powered.

Example code:

```

local input = script.Parent.Input
local output = script.Parent.Output

input.SourceValueChange:Connect(function(value)
    if value >= 0.5 then
        for _, player in pairs(game.Players:GetPlayers()) do
            output.LastInteractedWith.Value = player
            output:SetValue(1)
            task.wait(0.1) -- Small delay between players
            output:SetValue(0)
        end
    end
end)

```

@r - Random Player

Selects one random player from the server.

Example code:

```

local input = script.Parent.Input
local output = script.Parent.Output

input.SourceValueChange:Connect(function(value)
    if value >= 0.5 then

```

```

    local players = game.Players:GetPlayers()
    if #players > 0 then
        local randomPlayer = players[math.random(1, #players)]
        output.LastInteractedWith.Value = randomPlayer
        output:SetValue(1)
        task.wait(0.5)
        output:SetValue(0)
    end
end
end)

```

@nin - Player Closest to Input

Finds the player nearest to the block that sent the input signal.

Example code:

```

local input = script.Parent.Input
local output = script.Parent.Output

input.SourceValueChanged:Connect(function(value)
    if value >= 0.5 then
        local inputBlock = input.Source.Parent -- The block that sent the
signal
        local closestPlayer = nil
        local closestDistance = math.huge

        for _, player in pairs(game.Players:GetPlayers()) do
            if player.Character and
player.Character:FindFirstChild("HumanoidRootPart") then
                local distance =
(player.Character.HumanoidRootPart.Position -
inputBlock.Position).Magnitude
                if distance < closestDistance then
                    closestDistance = distance
                    closestPlayer = player
                end
            end
        end

        if closestPlayer then

```

```

        output.LastInteractedWith.Value = closestPlayer
        output:SetValue(1)
        task.wait(0.5)
        output:SetValue(0)
    end
end
end)

```

@nou - Player Closest to Output

Finds the player nearest to the current block (the one with the output).

Example code:

```

local input = script.Parent.Input
local output = script.Parent.Output
local thisBlock = script.Parent -- The current block

input.SourceValueChanged:Connect(function(value)
    if value >= 0.5 then
        local closestPlayer = nil
        local closestDistance = math.huge

        for _, player in pairs(game.Players:GetPlayers()) do
            if player.Character and
            player.Character:FindFirstChild("HumanoidRootPart") then
                local distance =
                (player.Character.HumanoidRootPart.Position - thisBlock.Position).Magnitude
                if distance < closestDistance then
                    closestDistance = distance
                    closestPlayer = player
                end
            end
        end

        if closestPlayer then
            output.LastInteractedWith.Value = closestPlayer
            output:SetValue(1)
            task.wait(0.5)
            output:SetValue(0)
        end
    end
end)

```

```
end  
end)
```

@p[x] - Player Within Radius

Finds the closest player within a specified radius. Many creators use "@rad" with a separate NumberValue for the radius.

Example code:

```
local input = script.Parent.Input  
local output = script.Parent.Output  
local thisBlock = script.Parent  
local radiusValue = script.Parent:FindFirstChild("Radius") or  
Instance.new("NumberValue") -- Configurable radius  
radiusValue.Value = radiusValue.Value or 50 -- Default 50 studs  
  
input.SourceValueChanged:Connect(function(value)  
    if value >= 0.5 then  
        local closestPlayer = nil  
        local closestDistance = math.huge  
        local maxRadius = radiusValue.Value  
  
        for _, player in pairs(game.Players:GetPlayers()) do  
            if player.Character and  
player.Character:FindFirstChild("HumanoidRootPart") then  
                local distance =  
(player.Character.HumanoidRootPart.Position - thisBlock.Position).Magnitude  
                if distance <= maxRadius and distance < closestDistance  
then  
                    closestDistance = distance  
                    closestPlayer = player  
                end  
            end  
        end  
  
        if closestPlayer then  
            output.LastInteractedWith.Value = closestPlayer  
            output:SetValue(1)  
            task.wait(0.5)  
            output:SetValue(0)  
        end  
    end  
end)
```



```
        end
    end
end)
```

@g - Gate Owner

Targets the player who owns the block. The OwnerId is automatically created by the game.

Example code:

```
local input = script.Parent.Input
local output = script.Parent.Output
local blockModel = script.Parent.Parent -- The main block model
local ownerIdValue = blockModel:WaitForChild("OwnerId") -- Created by the
game

input.SourceValueChanged:Connect(function(value)
    if value >= 0.5 then
        local ownerId = ownerIdValue.Value
        local owner = game.Players:GetPlayerByUserId(ownerId)

        if owner then
            output.LastInteractedWith.Value = owner
            output:SetValue(1)
            task.wait(0.5)
            output:SetValue(0)
        end
    end
end)
```

Variables

Variables in this game work just like variables in programming: they represent a number or piece of data that can be stored, updated, and displayed later.

Using Variables in Text

You can display a variable inside a text string by using the <%variableName> syntax.

- **Example:** If you have a variable called "apple" with a value of 5, writing "I have <%apple> apples!" will display as "I have 5 apples!".

Note: For more information on how text and strings work in Roblox, check out the [Roblox String Documentation](#).

Setting Up Variables

To use variables in your mod, you must first ensure the global variable table exists. Add this to the top of your script:

```
if _G["userVars"] == nil then
    _G["userVars"] = {}
end

-- This function finds variables hidden inside
your text strings
local function parseText(text)
    local foundVars = {}
    local token = string.find(text, "<%%")
    while token do
        local close = string.find(text, ">",
token)
        if close then
            local varName = string.sub(text,
token + 2, close - 1)
            foundVars[varName] = true
            token = string.find(text, "<%%",
close)
        end
    end
    return foundVars
end
```

Displaying Variables (Parsing)

When you want to turn the <%variableName> text into actual numbers, use this logic.

Disclaimer: This works like a command block; it updates once when triggered, it does not live-update like a sign.

```
local textstring = YourStringValue.Value
local foundVars = parseText(textstring)

for varName, _ in pairs(foundVars) do
    local userVars = _G["userVars"]
    local var = userVars[varName]
    -- Replace the tag with the variable value (or 0 if it
    doesn't exist)
    textstring = string.gsub(textstring, "<%%" .. varName .. ">",
    var and var.value or 0)
end
```

Creating and Updating Variables

Use the function below to change the value of a variable. This will also "fire" an event so that other objects in the game (like signs) know they need to update.

```

local function updateVariable(variableName,
Value)
    local userVars = _G["userVars"]
    local var = userVars[variableName]

    if var == nil then
        -- Create a new variable if it doesn't
exist
        local event =
Instance.new("BindableEvent")
        event.Name = "userVarChangedEvent_" ..
variableName
        event.Parent = game.ServerStorage

        var = {value = 0; event = event; name =
variableName}
        userVars[variableName] = var
    end

    var.value = Value
    var.event:Fire(var.value) -- Tell the game
the value changed
end

-- Example usage:
updateVariable("MyVariable", 5)

```

Recommended Plugins for Modkit Development

DWeld

What it does: Automatically welds all parts to a model's PrimaryPart **Why you need it:** Essential for creating unanchored props, and blocks you want to be able to be affected by physics

Get it here: <https://create.roblox.com/store/asset/5019960405/DWeld-Weld-Models>

Archimedes

What it does: Advanced building tools with precise rotation and alignment **Why you need it:** Makes complex part positioning much easier

Get it here: <https://create.roblox.com/store/asset/144938633/Archimedes-v319>

Alignment+

What it does: Improved part alignment and snapping tools **Why you need it:** Speeds up block assembly with better precision

Get it here:

<https://create.roblox.com/store/asset/6230964447/Alignment-PartModel-AlignerResizer-REMADE>

Stravant Plugin Series

What it does: Collection of useful building and editing tools **Why you need it:** Quality-of-life improvements for Studio workflow

Get them here: Search "Stravant" in the Toolbox/Creator Store

Quick Checklist Before Publishing

- ☐ Deprecated objects are enabled in Studio settings
- ☐ Every block has a unique 4-9 digit AssetId
- ☐ All blocks have a Model or BasePart named "Model"
- ☐ Primary parts are invisible, non-collidable, and it's size is a multiple of 4
- ☐ Primary parts are set as PrimaryPart in their Model
- ☐ Physics blocks are fully welded and unanchored
- ☐ Subcategories are properly configured with SubtypeOfId and Semantic
- ☐ Configuration objects are named "Configuration"
- ☐ Dropdown BoolValues are all set to true
- ☐ CustomEvents and CustomEventReceivers are properly scripted

- ☐ Wiring identifiers are coded if your blocks use them
-

Common Mistakes

- ✗ **Duplicate AssetIds** - Breaks your entire modkit
 - ✗ **AssetId too large (Larger than 2^{53})** - Makes blocks unsaveable
 - ✗ **Model not named "Model"** - Block becomes unplaceable
 - ✗ **Primary part size not multiple of 4** - Placement issues
 - ✗ **Forgetting to set PrimaryPart** - Game creates wrong hitbox
 - ✗ **Dropdown BoolValues set to false** - Options don't appear
 - ✗ **Forgetting to enable Show Deprecated Objects** - Deprecated objects won't appear when you try to insert them in the explorer
-

Happy modkit making! 🎮

For questions and support, visit the bild island community.